Handrail user guide

How to install it, run the tests, drive the mock bank, discover a capability once, replay it, escalate to a person, let an agent order, read the evidence, and record a run. Written 2026-10-11 for the interface.ai take-home. Repository: github.com/BillDhawal/handrail

1. Install
2. Run the tests
3. The mock bank, PLUMBLINE
4. Discover once
5. Replay
6. Verify and approve
7. Escalation and the console
8. An agent orders through MCP
9. Evidence
10. Recording a run
11. Troubleshooting

1. Install

Python 3.12 and **uv**. The core needs only pydantic; everything else is an opt-in extra. The browser extra brings Playwright and a Chromium download; author brings LangChain; classify brings the on-device referee Laya and the TypeSafe SDK; serve brings the MCP server and the example agent.

```
git clone https://github.com/BillDhawal/handrail && cd handrail
uv sync --extra browser --extra author --extra classify --extra serve
uv run playwright install chromium
cp .env.example .env     # put ANTHROPIC_API_KEY=sk-ant-... in it
```

Keys. `ANTHROPIC_API_KEY` is used by discovery, the Claude referee and the scout. **Replay needs no key.** Laya runs on-device and needs none; its checkpoint downloads on first use, about a minute. `TYPESAFE_API_KEY` is optional. The `.env` file is gitignored.

2. Run the tests

```
make test
```

```
uv run ruff check src tests targetapp
All checks passed!
uv run ruff format --check src tests targetapp
118 files already formatted
uv run mypy
Success: no issues found in 57 source files
uv run pytest -q
.........  (333 dots)
333 passed in 16s
```

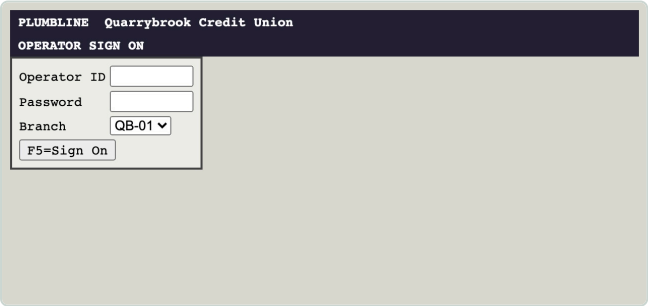
No key and no server are needed. Most tests run the engine on a scripted surface; seventeen drive a real headless Chromium against PLUMBLINE started inside the test on a free port; the authoring loop runs with a scripted fake model; one records a short video. `make lint` fixes formatting and never changes behaviour. One file: `uv run pytest tests/replay/test_engine.py -vv`. Read first: `test_a_step_in_doubt_is_never_repeated_across_runs`, the double-post bug from the prototype, made impossible by the journal.

3. The mock bank, PLUMBLINE

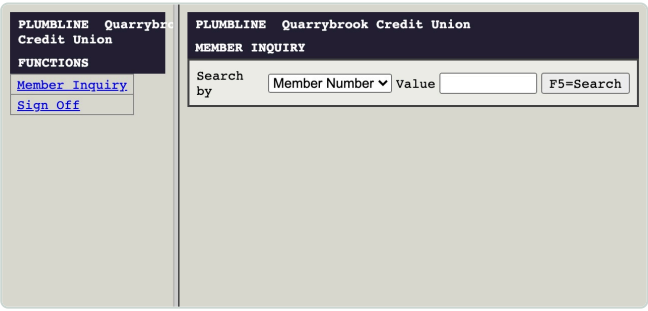
A Flask app in `targetapp/`, deliberately hostile the way real core-banking screens are: framesets, no ids, no labels, table layout, a per-render form token, terse messages. Two tenants on two ports with different markup. Faults can be armed on a test endpoint.

```
make up           # Quarrybrook on :8081, Fernhollow on :8082
make down
export BASE_URL=http://127.0.0.1:8081
```

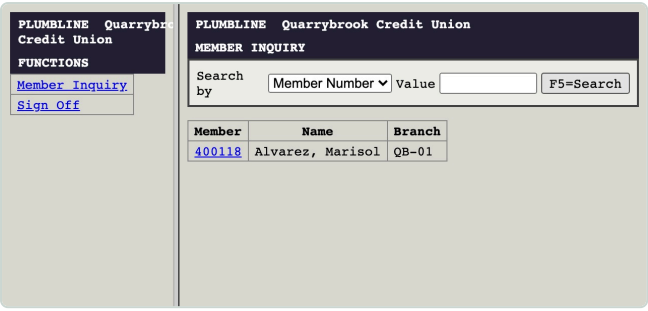
Sign on as `dcolewell` / `plumbline-demo`. Members 400118, 400226, 400337, 400445. Share 400226-S0002 is already on hold. The place-hold flow is seven screens:



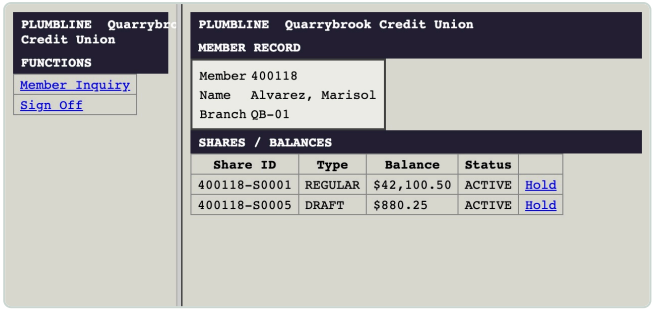
1. Sign on. Fields have no labels: the caption sits in the cell to the left.



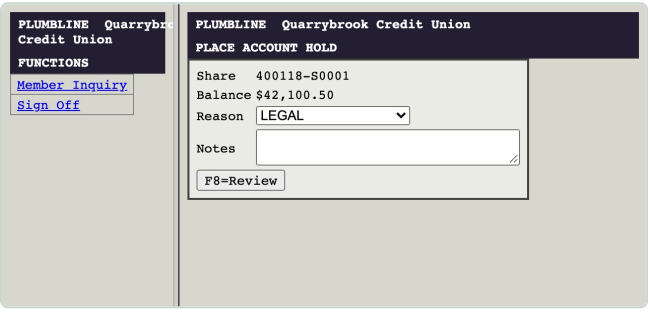
2. The desk: a frameset, menu on the left, inquiry on the right.



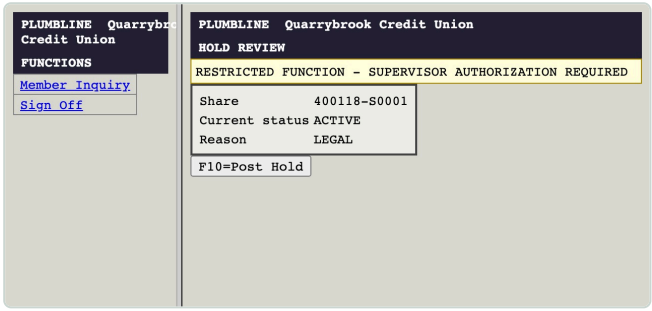
3. Inquiry results: the member link is named by the member number.



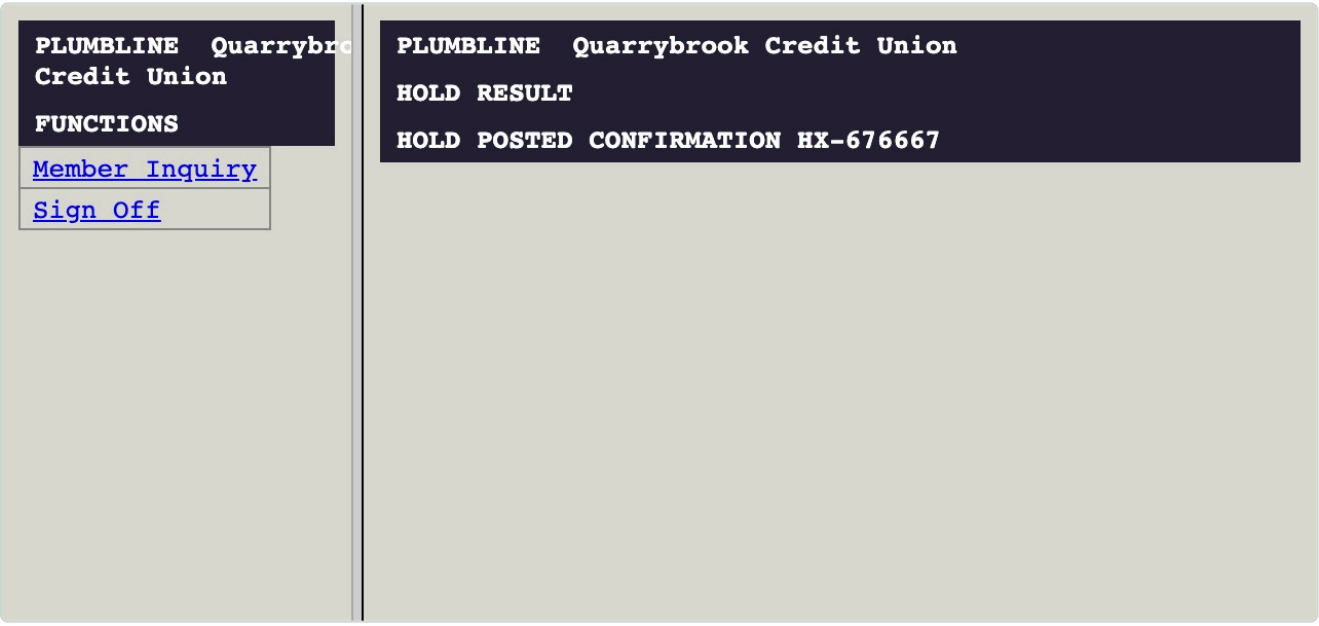
4. The member record: one Hold link per share, all named Hold.



5. The hold form: reason and notes.



6. Review, with the one commit on the flow: F10=Post Hold.



7. The result. Posted and already-held render the same template; they differ by one class on the message line.

Faults: `slow`, `dialog`, `maintenance`, `error`, `drift_minor` (a banner on every page), `drift_reword` (the result line reworded and reclassed), `drift_major` (the result page rewritten). Arm one, reset, and ask the record store:

```
curl -s -X POST -H 'Content-Type: application/json' \
  -d '{"fault":"drift_reword","on_path":"/","once":false}'
http://127.0.0.1:8081/__test__/arm_fault
curl -s -X POST http://127.0.0.1:8081/__test__/reset
curl -s http://127.0.0.1:8081/__test__/share/400118-S0001     # the record store, not the
screen
```

4. Discover once

A model drives the bank through a numbered menu of legal operations with three tools: `act(index, verb, value)`, `assert_screen(label)`, `finish(outcome)`. It never sees a selector. Values are blanks such as `{{input.password}}`; the real value never enters the trace. Every button press asks you at the terminal whether it is a commit.

```
uv run handrail author \
  --goal "Sign on as the operator, search for the member by member number, open the member
record, open the hold form for the given share, choose the hold reason, go to the review
screen, post the hold, and finish on the result screen." \
  --id plumbline.place_hold --entry http://127.0.0.1:8081/signon --entry-template
'{{env.BASE_URL}}/signon' \
  --allow 127.0.0.1:8081 --out capabilities/plumbline-place-hold.authored.json \
  --input operator_id=dcolewell --input password=plumbline-demo --secret password \
  --input member_number=400118 --input share_id=400118-S0001 --input reason=LEGAL --by
"$USER"
```

```
commit? row 7: invoke button 'F5=Sign On'  [c]ommit / [n]avigate / [s]tage / [d]eny: n
...
turns=19  model_calls=12  finished=True  outcome=Hold Result
evidence=evidence/runs/author_20261008T044658Z_52dae1
compiled plumbline.place_hold@0.1.0: 11 steps, 7 screens, draft -> capabilities/plumbline-
place-hold.authored.json
```

Add `--confirm commit` to answer every prompt up front; your name is written into the card as the confirmer. The compiled card is a **draft**. The run's book is `trace.jsonl` in its evidence directory, one line per accepted turn, password masked.

5. Replay

Inputs in, a typed result out, no model. Exit code 0 success, 2 business outcome, 3 recoverable, 4 hard failure.

```
uv run handrail replay capabilities/plumbline-place-hold.authored.json \
  --input operator_id=dcolewell --input password=plumbline-demo \
  --input member_number=400118 --input share_id=400118-S0005 --input reason=LEGAL
```

```
SUCCESS  code=NONE  outcome=Hold Result
outputs={}
llm_calls=0  classifier_calls=0  drift=11/11 first-choice
steps=11  duration_ms=747  evidence=evidence/runs/replay_20261008T...
```

A share already on hold is an answer, not a failure. The hand-written card declares it, and `no_match` too:

```
uv run handrail replay capabilities/plumbline-place-hold.json \
  --input operator_id=dcolewell --input password=plumbline-demo \
  --input member_number=400226 --input share_id=400226-S0002 --input reason=LEGAL
```

```
BUSINESS_OUTCOME  code=ALREADY_PROCESSED  outcome=already_held
```

category	meaning	retryable
SUCCESS	done; outputs are in the result	no need
BUSINESS_OUTCOME	the application said no: ALREADY_PROCESSED, RECORD_NOT_FOUND	no
RECOVERABLE	MISSING_CONTROL, SLOW_LOAD, UNEXPECTED_DIALOG, SESSION_EXPIRED, TRANSIENT_FAILURE	yes
HARD_FAILURE	INVALID_INPUT, SCREEN_MISMATCH, TARGET_MISMATCH, POLICY_VIOLATION, UNSAFE_TO_RETRY, ABORTED_BY_OPERATOR	no

Other flags: `--headed` shows the browser; `--journal PATH` shares one journal across runs so a commit in doubt stays in doubt; `--video DIR` records the browser; `--classifier`, `--bridge` and `--console` are the ladder, section 7.

6. Verify and approve

Three clean replays from a cold start for three different members, each confirmed by an independent read of the record store, never the screen. Then a named person signs. Only approved cards are served to agents.

```
W=operator_id=dcolewell,password=plumbline-demo
uv run handrail verify capabilities/plumbline-place-hold.authored.json --env
BASE_URL=$BASE_URL \
  --trial "$W,member_number=400118,share_id=400118-S0005,reason=LEGAL" \
  --trial "$W,member_number=400337,share_id=400337-S0001,reason=LEGAL" \
  --trial "$W,member_number=400445,share_id=400445-S0001,reason=FRAUD_REVIEW"
uv run handrail approve capabilities/plumbline-place-hold.authored.json --by "$USER"
```

```
plate 1: clean
plate 2: clean
plate 3: clean
verified plumbline.place_hold@0.1.0 after 3 clean plates -> capabilities/plumbline-place-
hold.authored.json
approved plumbline.place_hold@0.1.0 by dhawal at 2026-10-08T05:31:04+00:00
```

A card whose commit nobody confirmed cannot leave draft, however clean the plates. A draft cannot be approved. Reset the bank between tastings; a held share stays held.

7. Escalation and the console

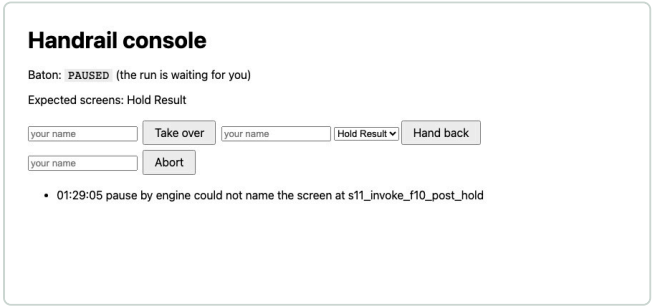
When a screen check fails, the run climbs a ladder, cheapest first. Each rung only proposes; the engine checks the proposal against the page's structure before believing it.

rung	who	flag	check before acting
1	a referee: Claude, Laya (on-device), or Jev	`--classifier claude|laya|jev`	confidence at or above 0.80 with margin 0.20, and the chosen screen's stored furniture overlaps the page at 0.5
2	a scout: the authoring model with commit struck off, four turns	`--bridge anthropic:claude-sonnet-5`	must name an expected screen, never one behind the run; same furniture check
3	you, at the console, on the same live browser	`--console 8765 --headed`	the name must be a screen the run was hoping for

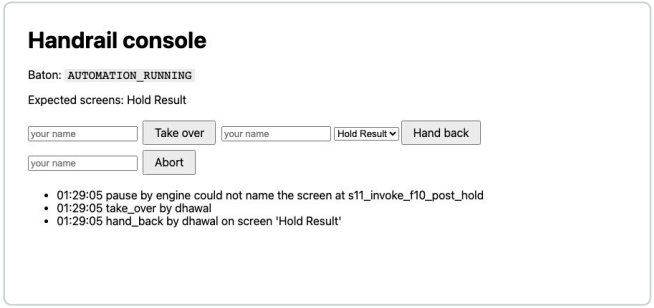
```
# a reworded result line: the referee places it
uv run handrail replay capabilities/plumbline-place-hold.authored.json --classifier claude --
episodes episodes.db \
  --input operator_id=dcolewell --input password=plumbline-demo \
  --input member_number=400337 --input share_id=400337-S0001 --input reason=LEGAL
uv run handrail episodes episodes.db        # the referee's calibration table

# the result page rewritten: it comes to you
uv run handrail replay capabilities/plumbline-place-hold.authored.json --classifier claude --
console 8765 --headed \
  --input operator_id=dcolewell --input password=plumbline-demo \
  --input member_number=400445 --input share_id=400445-S0001 --input reason=LEGAL
```

```
console: http://127.0.0.1:8765/
... "event": "rung1.screen", "chosen": "none_of_these", "confidence": 0.95, "accepted": false
... "event": "rung3.paused", "expected": ["Hold Result"]          <- the run is waiting for
you
```



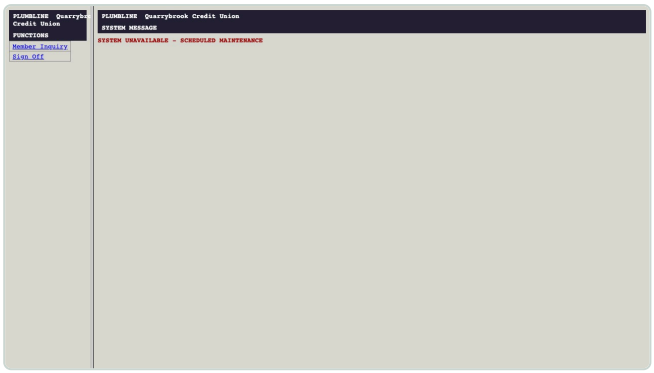
The console while the run waits. Type your name, Take over, fix things in the headed browser.



After Hand back naming the screen. Every exchange of the baton is listed with a name and a reason.



What the person saw at takeover in the evidence set: the result page rewritten entirely.



A failure's photograph: a maintenance page armed on the hold form. The run ended RECOVERABLE SLOW_LOAD.

```
SUCCESS  code=NONE  outcome=Hold Result
llm_calls=0  classifier_calls=1 ...        result.json: "escalated_to_human": true
```

8. An agent orders through MCP

One typed tool per approved card. The agent never sees a screen or a password; the server fills secrets from its own environment.

```
HANDRAIL_SECRET_PASSWORD=plumbline-demo uv run python examples/agent_caller.py \
  "Place a LEGAL hold on share 400226-S0002 of member 400226"

uv run handrail serve --capabilities capabilities     # the server itself, MCP over stdio
```

```
tools on the menu: ['plumbline_place_hold']
tool calls: 1
  receipt: {"category": "BUSINESS_OUTCOME", "code": "ALREADY_PROCESSED", "outcome":
"already_held", "retryable": false, ...}
agent says: The request has already been handled; share 400226-S0002 already has a hold in
place. No retry is warranted.
```

Any MCP client can be pointed at `uv run handrail serve --capabilities capabilities` with `HANDRAIL_SECRET_PASSWORD` and `BASE_URL` in its environment.

9. Evidence

Every run writes a directory under `evidence/runs/` (gitignored): a hash-chained `log.jsonl`, `result.json`, the journal, and on failure a screenshot and the page text. `evidence/examples/` is the committed set: the real discovery run and seven replays, indexed in its README.

```
uv run python -c "from handrail.kernel.evidence import verify_chain; from pathlib import
Path; print(verify_chain(Path('evidence/examples/1-replay-success/log.jsonl')))"
# (True, 0) means intact; (False, n) names the first edited line
```

Reading a log line: `event` is the moment (`step.start`, `step.ok`, `screen.similar`, `rung1.screen`, `rung3.paused`, `replay.error`); `prev` and `hash` are the chain. A `replay.error`

line says the step, what was expected, what was seen, and names the screenshot.

10. Recording a run

The browser records itself

Add `--video DIR` to `replay` or `author`. Playwright writes one `.webm` into the directory when the run closes. It works headless, so it is the clean option for a submission.

```
uv run handrail replay capabilities/plumbline-place-hold.authored.json --video recordings \
  --input operator_id=dcolewell --input password=plumbline-demo \
  --input member_number=400118 --input share_id=400118-S0005 --input reason=LEGAL
ls recordings/                                                  # one .webm
ffmpeg -i recordings/*.webm -c:v libx264 -pix_fmt yuv420p run.mp4    # optional: brew install
ffmpeg
```

The whole screen, terminal and browser together

On macOS press `Cmd`+`Shift`+`5`, choose Record Selected Portion, draw a box around the terminal and the headed browser, press Record; stop from the menu bar. The file lands on the Desktop as `.mov`. Run with `--headed` so the browser is visible, and for the console scenario keep the console page in a third window. QuickTime Player, File, New Screen Recording, does the same with a microphone if you want to narrate.

A two-minute script: `make test` finishing green; one replay with its receipt; the already-held replay; arm `drift_major`, replay with `--console 8765 --headed`, take over, hand back; the agent example. The brief calls a recording welcome but optional.

11. Troubleshooting

symptom	cause and fix
sign-on rejected on replay though the password is right	a stale PLUMBLINE from an earlier session still owns port 8081: `make down`, then `lsof -i :8081`
`401 API key is invalid`	the key in `.env` was rotated or mangled; paste a fresh one
the referee is "unavailable" in the log	no key for Claude or Jev; or Laya's checkpoint did not fit on disk (a few GB free)
an MCP client reports "Connection closed"	something printed to stdout in the server process; stdout is the wire
SLOW_LOAD under drift, no referee call	the card has no stored screen paths; re-author it, or fill `paths` from `handrail observe`
already held when you expected posted	the store remembers; `curl -X POST .../__test__/reset` between demos